Keystatic Demo

Release

Keystatic Demo Team

Table of Contents

1 Frequently Asked Questions **1**
1.1 Can I use MDX components? . . . 1
1.2 How do I add content? . . . 1
1.3 How does GitHub mode work? . . . 2
1.4 What is Keystatic? . . . 2

2 Release Notes **3**
2.1 v2.3.0 . . . 3
2.2 v2.3.1 . . . 3
2.3 v2.4.0 . . . 3
2.4 v3.0.0 . . . 4
2.5 v4.0.0 . . . 4
2.6 v4.0.0 . . . 4

Welcome to the documentation. This is auto-generated from content managed via Keystatic.

1 Frequently Asked Questions

1.1 Can I use MDX components?

Yes. Keystatic supports MDX out of the box, so you can embed React components directly in your content files. :party:

1.1.1 Example

```
;This is a custom callout rendered from a React component.
```

1.1.2 Things to keep in mind

- Components must be importable from the content file's location.
- The MDX field in Keystatic gives editors a rich-text toolbar, but raw component syntax is preserved as-is.
- If you need structured data instead of freeform markup, consider using Keystatic's `fields.object` or `fields.blocks` instead of MDX components.

1.2 How do I add content?

There are two ways to add content to your Keystatic-powered site.

1.2.1 Using the admin UI

1. Navigate to `/keystatic` in your browser.
2. Choose a collection (e.g. FAQ or Release Notes).
3. Click **Create** and fill in the fields.

4. Save — in GitHub mode this opens a pull request; in local mode the file is written immediately.

1.2.2 Editing files directly

Because Keystatic stores content as plain files, you can create or edit Markdown and MDX files in your editor of choice. Just make sure the frontmatter matches the schema defined in `keystatic.config.ts`.

1.3 How does GitHub mode work?

GitHub mode lets non-technical editors propose content changes through the Keystatic UI while keeping your repository as the single source of truth.

1.3.1 Setup

In `keystatic.config.ts`, set the storage kind to `github` and provide your repo:

```
storage: {
  kind: "github",
  repo: "owner/repo-name",
  branchPrefix: "content/",
}
```

1.3.2 What happens when an editor saves?

1. Keystatic creates a new branch prefixed with `content/`.
2. The content change is committed to that branch.
3. A pull request is opened against the default branch.
4. Your team reviews and merges the PR like any other code change.

This means every content edit goes through version control and can be rolled back at any time.

1.4 What is Keystatic?

Keystatic is a content management tool built for developers who want a Git-based workflow without giving up an editing UI.

1.4.1 How does it work?

Keystatic reads and writes content as files — Markdown, MDX, JSON, or YAML — directly in your repository. Editors get a polished admin interface at `/keystatic`, while developers keep full control over schemas, rendering, and deployment.

1.4.2 Do I need a database?

No. All content lives in your repo. In local mode, changes write straight to disk. In GitHub mode, every edit creates a pull request so content goes through the same review process as code.

1.4.3 What frameworks does it support?

Keystatic works with **Astro**, **Next.js**, and **Remix** out of the box. Any framework that can read local files can integrate with Keystatic's reader API.

2 Release Notes

2.1 v2.3.0

2.1.1 Highlights

- **Blocks field** — A new polymorphic array field that lets editors build pages from different block types (rich text, callouts, CTAs, embeds).
- **Conditional fields** — Fields can now change their schema based on a discriminant select, enabling template-style entry forms.
- **Multi-relationship field** — Reference multiple entries from another collection with optional length validation.

2.1.2 Breaking changes

- The `fields.document` field has been renamed to `fields.mdx`. A codemod is available: `npx @keystatic/codemods document-to-mdx`.

2.1.3 Bug fixes

- Array fields now preserve item order after drag-and-drop reordering.
- Fixed TypeScript types for the `fields.object` helper when used inside arrays.

2.2 v2.3.1

2.2.1 Bug fixes

- **GitHub mode**: Fixed an error when creating branches on repositories with protected default branches.
- **Reader API**: `collection.all()` no longer throws when the content directory is empty.
- **Admin UI**: Corrected tab order in the entry editor for improved keyboard accessibility.

2.3 v2.4.0

2.3.1 Highlights

- **MDX field toolbar** — The content editor now includes formatting buttons for bold, italic, links, and code blocks directly in the MDX field.
- **Date field defaults** — Date and datetime fields can now be initialized to `today` or `now` using a `defaultValue` option.
- **Improved slug validation** — Slug fields accept a `pattern` option with a custom regex and error message.

2.3.2 Bug fixes

- Fixed an issue where multiselect defaults were not applied on new entries.
- Resolved a rendering glitch in the image field preview on Safari.
- The reader API now correctly resolves linked files in nested objects.

2.4 v3.0.0

2.4.1 Highlights

- **Astro 6 support** — Full compatibility with Astro 6, including the new content layer API and server islands.
- **S3 storage adapter** — A new storage backend that reads and writes content to an S3-compatible bucket, enabling deployments without Git access.
- **Improved admin performance** — The entry list loads up to 3x faster on collections with more than 500 entries thanks to virtual scrolling.
- **Dark mode for the admin UI** — The Keystatic admin panel now respects `prefers-color-scheme` and includes a manual toggle.

2.4.2 Migration guide

1. Update all `@keystatic/*` packages to `^3.0.0`.
2. Replace `fields.document` with `fields.mdx` if you haven't already (deprecated since v2.3.0).
3. If using a custom reader, update imports from `@keystatic/core/reader` — the `createReader` signature now requires a storage config object instead of a base path.

2.4.3 Bug fixes

- Fixed hot-reload losing unsaved changes in the admin UI during development.
- GitHub mode now supports repositories with more than 1,000 files in a single directory.

2.5 v4.0.0

2.6 v4.0.0

2.6.1 Keystatic GitHub OAuth on Cloud Hub

This release enables Keystatic's GitHub authentication to work correctly when deployed on Cloud Hub.

Changes

Middleware for Proxy URL Rewriting

Added Astro middleware (`src/middleware.ts`) to handle reverse proxy URL rewriting. Cloud Hub's reverse proxy routes requests to containers, but the internal `Host` header shows `localhost:3000`. The middleware rewrites the request URL using:

- `x-forwarded-host` and `x-forwarded-proto` headers (if available)
- Falls back to `SITE_URL` environment variable

This ensures Keystatic generates the correct OAuth `redirect_uri` pointing to the Cloud Hub domain instead of localhost.

Cloud Hub Egress Configuration

Added egress labels to `docker-compose.yml` to allow outbound traffic to GitHub:

```yaml
labels:
  - 'com.calponia.egress.fqdns.0.exact=github.com'
  - 'com.calponia.egress.fqdns.1.exact=api.github.com'
```

Cloud Hub blocks all outbound internet traffic by default. Without these rules, the container cannot reach GitHub's API to exchange OAuth authorization codes for access tokens.

Debug Endpoints (Development)

Added temporary debug endpoints for troubleshooting:

- `/api/debug-env` - Verifies environment variables are correctly set
- `/api/test-github-token` - Tests GitHub API connectivity

Environment Variables Required

Variable	Description
`SITE_URL`	Full URL of the deployed site (e.g., `https://xxx.app.cloudhub.etas.com`)
`KEYSTATIC_GITHUB_CLIENT_ID`	GitHub OAuth App client ID
`KEYSTATIC_GITHUB_CLIENT_SECRET`	GitHub OAuth App client secret
`KEYSTATIC_SECRET`	Secret for session encryption (64 hex chars)
`PUBLIC_KEYSTATIC_GITHUB_APP_SLUG`	GitHub App slug name

Key Learnings

1. **Build-time vs Runtime**: Astro's `site` config is evaluated at build time. Pass `SITE_URL` as a build argument and set it in the Dockerfile.
2. **Cloud Hub Networking**: Containers have no internet access by default. Whitelist required domains via egress labels.
3. **Reverse Proxy Headers**: Don't assume `x-forwarded-*` headers are passed. Always have a fallback mechanism.